

```
> <CaveSurvey$Dir>.SU_2/03
|> <CaveSurvey$Dir>.u_su2/03 NB release number defined in FNrelease
```

```
Version 1.65 Last modified 1989.09.08
Version 1.66 1989.10.02 integrated into 2.00 1989.11.30
```

```
2.00 development version 1989.12.22 from BBC: master via HostFS
2.01 units now 10^-4 m 1990.05.13 because of big Austrian gridrefs
2.02 fix no HostVdu bugs 1990.07.25 also deglobalise I% somewhat
2.03 kill global residents 1990.10.04 change FNscan to PROCscan etc.
```

debug to HostVDU - note this will interleave with any other debugging info, eg. from the IconBar program or any IconCLI utilities.

Work started on Release 2 on 1989.09.19

Release 2 is to implement the Warren closure algorithms.
it will also take command line args, asking user only if forced to
at some stage it will run in a window.

Bug 1989.01 Documentation stated that *keyword was ignored if keyword was not recognised. In fact, it was treated as a FROM station.
this has now been fixed in PROCdata, but note that when * is compulsory, the fix should be in PROCinline.

Bug 1989.02 Plumbed legs were not included in the true length of a survey.
Now fixed in PROCdry.

Bug 1989.03 PROCdepth_gauge called, but PROCgauge DEFed, change DEF to fix
(these three fixes incorporated back into 1.67)

Criticisms ? Still rely rather too much on resident integers - no excuse for not having meaningful names throughout. Aim is to have NO resident integers except LOCAL ones.

GRADE keyword now processed and checked, and standard errors on x, y, z vectors calculated for all survey legs. The format of the vector (0.) file has changed - we only store the from/to internal numbers, not the actual vector x,y,z values which are only used in this program.
PROCnodes revised, N. file structure reduced.

(C) Pennine Software 1984, 1986, 1987, 1988, 1989, 1990

```
debug_device$ = "HostVdu:"
```

IDE7: *! you could use a prettier mode for A-machines.

```
ROCsetup
ROCfiles
cart_Time%=TIME
```

```
REPEAT:PROCinline:UNTILend%
```

```
ROCnodes
ROCTidyup
```

```
ROCloop_adjust : REM who are you kidding ?
```

```
ROCcoords
VD
```

```
IFFNrelease
IM time% 5
> no
```

```
FPROCloop_adjust
DPROC
```

```
FPROCskip(K%):LOCAL C%,J%
```

```
*****
```

```
skip : skip K% characters from input file OR point to <control_char>
       if it occurs before K% characters have been read
Locals : C% used to receive characters from file
         J% byte count within the field
```

```
1.40 : C% tested < 32 so <lf> can end lines, as per Twin or other A-editors
```

```
*****
```

```
= 0
PEAT
% = BGET#input%
% += 1
TIL C%<32 OR J%=K%
  C%<32 THEN PROCunread
DPROC
```

```
FPROCread_name(width%,K%):LOCAL C%,J%
```

```
*****
```

```
read_name : read a string of up to width%+1 characters into $K% ( padding to
              right with " " and leaving pointer pointing to <control_char> if it
              occurs before width%+1 characters have been read )
Parameters: width% : index of final character we can hold (chars 0..width%)
            K%      : address of buffer to hold string
Local      : J% used as a byte count within the field
            C% used to read characters into
```

```
1.40 : K%?J% tested < 32 so <lf> can terminate lines, as in Twin
1.63 : ignore leading asterisks for "Surveyor" compatibility issue
       actually, Sean has fucked about so this is hardly relevant...
```

```
*****
```

```
_a_keyword% = FALSE
= -1
PEAT
% += 1
% = BGET#input%
%?J% = C%
  remove this : IF J%=0 THEN J%=(C%=ASC("*"))  ;*! depends on representation of TR
  = (J%=0) AND (C%=ASC("*")) THEN J%=-1:is_a_keyword%=TRUE
TIL C%<32 OR J%=width%
  C%<32 THEN PROCunread:K%?J%=32

?cr%=13                ;*! just in case the delimiter wasn't a <cr> !

J%=final% ENDPROC      ;*! even if J%=width% we may need to pad !

} J% = J%+1 TO final%
?J%=32
{
}PROC
```

```
FNreadno(K%,convert%):LOCAL N$,N,C%,J%
```

```
*****
```

```
readno : read a number from a K% character field ( if <control_char> occurs
           before K% characters have been read, leave pointer pointing at
           <control_char> ) such that a blank or null-length field gives
           result zero.
```

If convert% has bit 0 set then convert to ~~deca-microns~~ ^{micro-meters} ✓
 If convert% has bit 1 set then return as a real else an integer
 If convert% has bit 2 set and conflag%=4 then convert units

Locals : N\$ accumulates characters from the line, then used to be EVALled
 N holds the real result of the conversion
 C% used to receive BGETted characters from the file
 J% used as a byte count within the field

1.40 : C% now tested <32, >31 to allow lines terminated by <lf>, as per Twin

```

*****
; = ""
; = BGET#input%
; C%<32 THEN PROCunread:=0
; = 1
:PEAT
N$ += CHR$(C%)
C% = BGET#input%
J% += 1
:UNTIL C%<32 OR J%=K%
; C%>31 N$ += CHR$(C%) ELSE PROCunread
; N$ = STRING$(LENN$, " ") THEN =0
=VAL(N$)
;convert%AND1 THEN

    N=DPfix%*N
:ENDIF
;convert%ANDconflag% N==convert ✓
;convert%AND2 THEN =N
[NT(N+0.5)

:FFNrest:LOCAL char%
*****
|
| rest : return the rest of the line up to any control character as a string.
|         note that if the pointer was pointing at <control_char> this is a
|         null string.
| Exit:   The file pointer points at the start of the next line
|
| Local   : char% used to receive characters from the file
| Features : No checking for excessively long lines, so could trap with
|             "String too long"
|             Terminated by any control character - essential for ARM versions
|
*****
rest$ = ""
char% = BGET#input%
; char%<32 THEN =""
:PEAT
rest$ += CHR$char%
char% = BGET#input%
:UNTIL char%<32
rest$

:FFNyn:LOCAL C%
*****
|
| FNyn : wait for user to press Y or N in either case. Return TRUE if it was Y
| Local : C% used as a result register for the INSTRing
|
*****
:PEAT
C% = INSTR("yYnN",GET$)
:UNTIL C%<>0
  
```

FPROCsetup

```

setup : define fixed constants, title, keywords, flags, offsets, and create
        machine code support.

Globals : RELEASE      used to print identity info at various stages
        end%           remains FALSE until program is to finish
        KEYS%          points to buffer holding valid keywords
        text1%         points to buffer for survey station name from or key
        text2%         points to buffer for survey station name to or subkey
        a%             buffer for up to 4 integers
        STAX           points to buffer for survey station table
        oldI$          string array to hold names of temporarily closed files
        oldP%          integer array to hold their PTR# values
        UWATER%        flag controls whether data are for dry or diving survey
        convert%       controls which conversions are to be done on data
        convert        controls factor by which data are to be converted
        DPfix%         scales internal units to metres
        conflag%       bit 2 set if convert is non-unity, zero otherwise
        DTHETA         amount to be SUBTRACTED from bearings to get true North
        DPHI           amount to be subtracted from clino reading to get truth
        surface_depth% and gauge_factor, similar for depth gauges
        Origins%       flag to control calling of PROCinitial_origin
        planlen%,
        sumlen%        accumulative total and plan lengths of survey
        rest$, COMMENT$ - used throughout to get strings from file

```

Lots of manifest constants which should not be compiled as variables :
 most define things that would appear as structures in a real language

LEASE=FNrelease

OCinitTUBEcommentary : REM but still safe if you don't have one

M args\$(100)

OCAssemble_Shell : REM -- This will create a GLOBAL var Shell_Env\$

```

nber_of_arguments% = FNShell_Array(args$(),Shell_Env$)
number_of_arguments% holds the number of command line tokens
args$()             contains the tokens
NOTE: The filename is NOT given (it is stripped off)

```

```

ze%      = 12      :%! total size of station field
size%    = 4
ize%     = 12      :%! size of a vector structure
ize%     = 12      :%! size of coordinate structure
ize%     = 12      :%! size of standard error vector structure
ASSERT Csize% = Esize% (so we can reuse the memory)
ne%      = 9       :%! DIM size for name field
%        = 8       :%! offset to delimiter
nal%     = 7       :%! offset to last alphabetic in name
fs%      = 9       :%! offset to reference count
rades%   = 11      :%! DIM size for list of survey grades
/        = 0       :%! Length \
/        = 1       :%! Bearing \
/        = 2       :%! Clino or > Warrenian standard error array indices
/        = 2       :%! Depth Gauge /
/        = 3       :%! Position /

```

```

1% = FALSE
> J% = 0 TO 1

```

- should have a local @% for this


```

:INT
declare survey station and keyword structures :
capacity% = 1500                                :! total number of stations we cope with
ea% = capacity%*size%
extent% = capacity% * FTsize%
extent% = capacity% * Vsize%
extent% = capacity% * Csize%
extent% = capacity% * Esize%
idth% = 5                                       :! until changed by WIDTH NAMES keyword
idth% = 6                                       :! NB default will become 7 at some stage
                                           :! until changed by WIDTH DATA keyword

M text1% name%,text2% name%,ax 16 3 has used?
M oldI$(4),oldP$(4)
M Afind% area%+size%-1 : STAX = Afind% + size%
M FTbuf% FTextent% : FTptr% = 0
M Vbuf% Vextent% : Vptr% = 0
M Coords% Cextent%
uf% = Coords% : Eptr% = 0
M grade_ids$(ngrades%), grade_spec(ngrades%,3)

default survey grades : (NB may prefer these in internal units ? )
ade_ids$(0) = "EN"
ade_spec( 0, dL% ) = 0.0
ade_spec( 0, dB% ) = 0.0
ade_spec( 0, dC% ) = 0.0
ade_spec( 0, dP% ) = 0.0
ade_ids$(1) = "5N"
ade_spec( 1, dL% ) = 0.1
ade_spec( 1, dB% ) = RAD(1.0)
ade_spec( 1, dC% ) = RAD(1.0)
ade_spec( 1, dP% ) = 0.1
ade_ids$(2) = "3N"
ade_spec( 2, dL% ) = 0.5
ade_spec( 2, dB% ) = RAD(2.5)
ade_spec( 2, dC% ) = RAD(2.5)
ade_spec( 2, dP% ) = 0.5
ade_ids$(3) = "DD"
ade_spec( 3, dL% ) = 0.5
ade_spec( 3, dB% ) = RAD(5.0)
ade_spec( 3, dD% ) = 0.5
ade_spec( 3, dP% ) = 0.5
R index% = 4 TO ngrades%
grade_ids$( index% ) = " "
OR index% = 0 TO 3
grade_spec( index%, index% ) = 0.
EXT
T
grade% = 3
ault_N_grade% = 1
ault_D_grade% = 3

index%=STAX TO STAX+area%-4 STEP4:!index%=0:NEXT
jindex%=0T04:oldI$(jindex%)=" "
":NEXT

1?cr%=13:text1%?refs%=0
2?cr%=13:text2%?refs%=0
ER%=FALSE
ert=1
ag%=0
%=10000
A=0.:DPHI=0.:planlen%=0:sumlen%=0
ce_depth% = 0: gauge_factor = 1
ns%=0

```

```

MMENT$=rest$
tle$=rest$      : *! reserve space in advance
tle$=""         : *! but default title is null
U28,0,23,39,2
DPROC

```

I swap lines

FPROCfiles

files : get filenames from user - check they exist (where possible)

```

Globals : path$      holds path name to reach SU subdirectories.
           note that this may be restricted by the filing system in use
survey$    subdirectory for primary input / all output
level%     current nesting level of subsidiary input files
input%     handle of open primary input file

```

ERROR PROClie_in_state:END

M we really want to do error box things under desk, but it will wait.

CAL AX

put% = 0

number_of_arguments% > 0 THEN

```

*! args$(0) should contain a filename or directory name. If its a file name,
*! then it should end in .D otherwise complain. If its a directory, then
*! the directory should contain a file .D else moan. If we are supporting
*! nested files, we expect that this directory has a sibling .Data, which
*! contains ALL the nested files.

```

survey\$ = FNsplitt_path(args\$(0))

IF survey\$=".D" OR survey\$="d" survey\$ = FNsplitt_path(args\$(0))

path\$ = args\$(0)

input% = FNcheck_openinput

DIF

if there were no command line arguments, then we end here as usual :

ILE input%=0

INPUT"Survey directory name : "path\$

INPUT"Which survey ? : "survey\$

input% = FNcheck_openinput

DWHILE

vel%=0

andle%=OPENOUT(path\$+survey\$+".0")

0handle%=0 PRINT""Unable to open ";path\$;survey\$;".0""Can't imagine why not.":E

0SE#0handle%:0handle%=0 : *! only opened out to check that we could

J12,26

RJX=0TO1:PRINTCHR\$141;CHR\$130:NEXT

J28,0,23,39,2

="" : *! to save trouble in nfile

DPROC

FPROCinline

inline : process a line from the current input file

Globals : text1% points to buffer for from station name or keyword

KEYS% buffer for keywords to check against "from" station

awidth% width of NAME field

~~Bug fix 1989.001 K% now completely irrelevant~~

EOF#input% PROCend:ENDPROC

OCread_name(awidth%,text1%)

3E \$text1% OF

WHEN "ORIGIN " : PROCorigin

```

WHEN "STYLE" " : PROCstyle
WHEN "CHANGE" " : PROCchange
WHEN "GAUGE" " : PROCdepth_gauge
WHEN "FILE" " : PROCnfile
WHEN "END" " : PROCend
WHEN "NOTES" " : PROCnotes
WHEN "WIDTH" " : PROCwidth
WHEN "TITLE" " : PROCTitle
WHEN "GRADE" " : PROCgrade
WHEN "FORMAT" " : PROCformat
WHEN "RADIO" " : PROCradiolocation
OTHERWISE : PROCdata

```

DCASE
DPROC

FFPROCend

```

end : process the END keyword in the input file
Globals : input%      handle of currently open input file
          I$          include-file leaf name (null for top level)
          level%      current depth of nesting - if zero then this is the
                      primary input file that we have open.
          end%        global flag to stop 1st phase of SU on closing primary
          K%          keyword index. will be / if we are here because of an
                      END keyword. will be some other value if the file just
                      ended on us.
BUG fix 1987.001 /    K% might have a value > -1 if the last line contained
                      a keyword. Condition changed to K%=>, from K%>0
BUG fix 1988.001 /    K% is completely irrelevant since inline uses CASE

```

```

USE#input%
input% = 0
level% = 0 end% = TRUE:ENDPROC
$TEXT1% = "END" " PRINT" "END occurs in nested file "path$;"Data.;"I$;"Halt pro
Nyn end% = TRUE:ENDPROC
level% += 1
I$ = oldI$(level%)
level% = 0 input% = OPENIN(path$+survey$+".D") ELSE input% = OPENIN(path$+"Data."+I$)
R#input% = oldP%(level%)
IDPROC

```

FFPROCnfile

```

nfile : save place in old file and open nested file for input
Globals : level%      current nesting level. limited arbitrarily to 4
          oldI$, I$    old and new include-file leaf names
          oldP%        PTR# of old file, so we can resume input later

```

```

level% > 4 PROCTHROWBOBBLER(1):ENDPROC
oldI$(level%) = I$
= FNrest
dP%(level%) = FNensurePTR(input%)
USE#input%
input% = 0
input% = OPENIN(path$+"Data."+I$)
input% > 0 level% += 1:ENDPROC

INT"Unable to open new input file" path$+"Data.;"I$
= oldI$(level%)
level% = 0 input% = OPENIN(path$+survey$+".D") ELSE input% = OPENIN(path$+"Data."+I$)

```

```

INT"Input continues from ";path$;
level% = 0 PRINTsurvey$;"D" ELSE PRINT "Data.";I$
IDPROC

FPROCorigin
*****

origin : read data defining alternate origin
Globals : text2%      station name for new origin is read into "to" buffer
          %X%, Y%, Z%  x,y,z coordinates of new origin (internal format)
          Nto%         index of new station name in existing list ( should be
could be LOCAL?  -1, if not, user is redefining an existing station )
          Nendsta%     internal number for new survey station
          $Ato%        new name added to list
          Ato%?refs%   count of references to new station - initially zero
          Origins%     Count of number of origins defined so far.

Note : 'Origin already defined' is now treated as fatal, so we always set up
      Nendsta% and Ato% explicitly after calling FNscan.

Bug fix 1987.003      Oendsta% assigned to valid value before calling FNscan.
                      I cannot see how the program ever worked before.

*****
OCskip(1+(nwidth%<<2)+awidth%)      :%! skip 4 data widths + one NAME width
OCcomment(132)
OCread_name(awidth%,text2%)          :%! new origin station name
OCskip(awidth%+1)                   :%! skip one NAME width
=FNreadno(12,5)                      :%! three coordinate fields
=FNreadno(12,5)                      :%! note coordinates are always 12 wide
=FNreadno(12,5)
Origins%=0 PROCinitial_origin(text2%,%X%,Y%,Z%):ENDPROC

Oendsta% = size%*Nendsta%            :%! Oendsta% is offset to end of station list
Ato% = FNscan(text2%)                :%! check station is not already on the list
OCscan( text2%, Ato%, Nto% )
IF Ato%>=0 PROCthrowwobbler(2) :%! and STOP execution !
  Nto%>=0 PROCthrowwobbler(2)
igins% += 1                          :%! increment origin count
Nendsta% += 1                        :%! increment station count
Ato% = Nendsta%*size%               :%! point to slot at end of list
$(STAX+Ato%) = $text2%              :%! record this station name
STAX?(Ato%+refs%) = 0               :%! no references yet
JCadd_station( text2%, Ato%, Nto%, 0 )

  record from station as -1 this leg
PROCrecord_from_to( -1, Nendsta% )
JCrecord_from_to( -1, Nto% )
JCrecord_vector( %X%, Y%, Z% )
JCrecord_error( 0, 0, 0 )           :%! standard errors on this leg are zero

JIgnore          :%! throw away rest of line
JPROC

FPROCrecord_from_to( Nfrom%, Nto% )
*****

*****
uf%!FTptr% = (Nfrom%<<16)+Nto%
>tr% += 4
JPROC

FPROCrecord_vector( %X%, %Y%, %Z% )
*****

```

```

*****
uf%!Vptr% = x%           : Vptr% += 4
uf%!Vptr% = y%           : Vptr% += 4
uf%!Vptr% = z%           : Vptr% += 4
DPROC

FPROCrecord_error( x%, y%, z% )
*****

*****
uf%!Eptr% = x%           : Eptr% += 4
uf%!Eptr% = y%           : Eptr% += 4
uf%!Eptr% = z%           : Eptr% += 4
DPROC

FPROCinitial_origin(Aorg%,X%,Y%,Z%)
*****

*****
TA% = $Aorg%
AZ?cr%=13
AZ?refs%=0               : *! No. of references
ndsta% = 0                : *! last (internal) station number
ndsta% = STAX             : *! pointer to last station
igins% = 1                : *! origin count
itialX% = X%
itialY% = Y%
itialZ% = Z%
INTCHR$130"ORIGIN at "CHR$134;$STA%
OCcomment(134)
DPROC

FPROCstyle
*****

style : process keywords to change type of input data
Globals :
    UWATER%      flag to indicate current style
    awidth%      width parm for NAMES. NB 1 less than actual width
    nwidth%      width parm for data

*****
JCread_name(awidth%,text2%)
3E $text2% OF

4EN "DIVING  "

JWATER%=TRUE
%ROCdive
%ROCskip(nwidth%<<1)

4EN "NORMAL  "

JWATER%=FALSE
%ROCskip(nwidth%<<2)

THERWISE

%ROCthrowwobbler(5)

```

DCASE

QCcomment(129)

DPROC

FPROCdive

```
dive : start surveying underwater type of data - get starting info
Globals : text2%      points to buffer for starting station name
          Anew%        internal station index for starting station
          depth        depth of most recently defined diving survey station
          why is this not in internal units?
Bug fix 1987.003      Qendsta% assigned before calling FNscan
Bug fix 1987.005      PROCthrowwobbler had wrong argument !
1.50                  non-fatal throwwobbles now numbered from 100 up
```

```
QCskip(nwidth%<<2)
QCcomment(129)
QCskip(awidth%+1)
QCread_name(awidth%,text2%)
Qendsta% = size%*Nendsta%
Anew% = FNscan(text2%)
JCscan( text2%, Anew%, Nnew% )
IF Anew%<0 PROCthrowwobbler(100)
   Nnew%<0 PROCthrowwobbler(100)
>th = FNreadno(nwidth%,7)
DPROC
```

FPROCchange

```
change : accept data to define new instrument calibrations
Globals : DTHETA      amount to be subtracted from compass to get true north
          DPHI        amount to be subtracted from clino reading to get truth
```

```
JCskip(1+nwidth%+awidth%)      :! skip one numeric and one NAME field
+ETA = FNreadno(nwidth%,2)
+I   = FNreadno(nwidth%,2)
JCskip(nwidth%)
JCcomment(134)
>PROC
```

FPROCdepth_gauge

```
gauge : accept data to define new depth gauge calibrations
Globals : surface_depth% amount to be subtracted from depth to get truth
          gauge_factor    amount to multiply gauge reading to get truth
```

```
JCskip(1+awidth%)      :! skip one NAME field
+face_depth% = FNreadno(nwidth%,5)
+ge_factor    = FNreadno(nwidth%,2)
JCskip(nwidth%<<1)
JCcomment(134)
>PROC
```

FPROCnotes

```
notes : print extra commentary from source data file
```

```
INT'FNrest
IDPROC
```

```
FPROCwidth: LOCAL oldwidth%
```

```
*****
```

```
width : process WIDTH keyword to set new field width
```

```
*****
```

```
OCread_name(awidth%,text2%)  :!* read subsidiary keyword field
dwidth% = nwidth%             :!* change doesn't happen till next line
```

```
SE $text2% OF
```

```
WHEN "NAMES "
```

```
awidth%=FNreadno(nwidth%,0)-1
IF awidth%>final% PROCthrowwobbler(4)
```

```
WHEN "DATA "
```

```
nwidth%=FNreadno(oldwidth%,0)-1
```

```
just ignore any other keyword
```

```
DCASE
OCskip(3*oldwidth%)
OCcomment(134)
DPROC
```

```
FPROCgrade
```

```
*****
```

```
grade : define precision specifiers for a survey grade, or define a
default grade to use for normal or diving data
```

```
*****
```

```
DAL index%
DCread_name( awidth%, text2% )
ade_id% = text2%?4
ade_type% = text2%?5
is_grade% = -1
R index% = 0 TO max_grade%
F (grade_id% = ASC( LEFT$(grade_ids$(index%))) ) AND (grade_type% = ASC( RIGHT$(gr
e% = index%
<T
ext2% = LEFT$( $text2%, 3 )
SE $text2% OF
```

```
WHEN "DEF"
```

```
IF (grade_type% <> ASC("D")) AND (grade_type% <> ASC("N")) PROCthrowwobbler(102)
IF this_grade% = -1 THEN
IF this_grade% = ngrades% PROCthrowwobbler(103):PROCignore:ENDPROC
this_grade% = max_grade%
max_grade% += 1
ENDIF
grade_spec( this_grade%, dL% ) = FNreadno( nwidth%, 6 )
grade_spec( this_grade%, dB% ) = RAD(FNreadno( nwidth%, 2 ))
IF grade_type% = ASC("D") grade_spec( this_grade%, dD% ) = FNreadno( nwidth%, 6
s%, dC% ) = RAD(FNreadno( nwidth%, 2 ))
grade_spec( this_grade%, dP% ) = FNreadno( nwidth%, 6 )
```

```
WHEN "USE"
```

```
IF this_grade% = -1 PROCthrowwobbler(104):PROCignore:ENDPROC
```

OTHERWISE

```
PROCthrowwobbler(105)
PROCignore
ENDPROC
```

```
DCASE
QCcomment(128)
DPROC
```

FFPROCformat

format : interpret FORMAT FIXED keywords, throw out FORMAT FREE keywords !

```
QCread_name(awidth%,text2%)
SE $text2% OF
```

```
HEN "FIXED  "
```

```
PROCskip(nwidth%<<2)
PROCcomment(128)
```

```
HEN "FREE  "
```

```
PROCthrowwobbler(8)
```

THERWISE

```
PROCthrowwobbler(7)
```

```
DCASE
DPROC
```

```
FPROCcomment(colour%)
```

comment : print trailing comment from input line with teletext code colour%

```
MENT$ = FNrest
LENCOMMENT$PRINT'CHR$(colour%)COMMENT$
DPROC
```

```
FPROCignore
```

ignore : throw away rest of line without processing

```
MENT$ = FNrest
DPROC
```

```
FPROCindicate_station
```

indicate_station : print the new(er) station name

```
POS>32 PRINT
Ato% > Afrom% PRINT;$text2%;" "; ELSE PRINT;$text1%;" ";
DPROC
```



```
*****
throwwobbler : handle errors
Globals : text2%      points to buffer holding erroneous station name
Locals  : EX          error number
          EX < 100 is a fatal error, so we stop
          unrecognised error numbers (careless !!) are treated as fatal

*****
INT'STRING$(80,"*"):VDU7,7
INT" Error on input"
SE EX OF
HEN 1 : PRINT"Input file nesting level too great""( maximum depth 4 )"
HEN 2 : PRINT"New origin redefines existing station : " ;$text2%
HEN 3 : PRINT"Depth change > leg length in dive"
HEN 4 : PRINT"New width " ;width% ;" for NAME field exceeds capability"
HEN 5 : PRINT"Unrecognised STYLE " ;$text2%
HEN 6 : PRINT"Unrecognised UNITS " ;$text2%
HEN 7 : PRINT"Unrecognised FORMAT parameter " ;$text2%
HEN 8 : PRINT"FREE FORMAT data entry not yet supported."

HEN 100 : PRINT"Diving survey start station " ;$text2%," undefined or missing"
HEN 101 : PRINT"UNITS CONVERT value silly or missing - taken as METRIC"
HEN 102 : PRINT"Survey grade type not N or D - ignored"
HEN 103 : PRINT"Too many survey grades defined - ignored"
HEN 104 : PRINT"Cannot default to undefined grade - ignored"
HEN 105 : PRINT"Undefined sub-key after GRADE keyword - ignored"
THERWISE : EX=0
PRINT"Unrecognised value " ;EX% ;" to throwwobbler.""Programmer must be going hoopy
DCASE
INT'STRING$(80,"*"):VDU7
EX<100 PROCcease ELSE ENDPROC

FPROCcease
*****

*****
Close all the files !!!
Ohandle% CLOSE#Ohandle%:PRINT"Closed Output file.":Ohandle%=0
input% CLOSE#input%:PRINT"Closed Input file.":input%=0
any others ?
JP

FPROCdata
*****

data : read and convert a survey leg - of either style
Globals : text1%, text2% pointers to buffers for from and to station names
          Afrom%, Ato%   pointers to from and to station names in main list
          L, THETA, D     values of true length, true bearing, plan length.
          X%, Y%, Z% x,y,z vector of current survey leg
          planlen%, sumlen%      cumulative plan and total length of survey

          1.52 : If no origin defined, take first station as (0,0,0)

*****
is_a_keyword% THEN
COMMENT$ = FNrest
?,"Warning, unrecognised keyword will not be processed : "
?,$text1% ;" " ;COMMENT$
ENDPROC
)IF
Origins%=0 PROCinitial_origin(text1%,0,0,0)
```

```

JClist(text1%,text2%)
JCindicate_station
  = FNreadno(nwidth%,7)
JETA = FNreadno(nwidth%,2) - DTHETA
J  = SINRAD(THETA)
J  = COSRAD(THETA)
  UWater% PROCdiving ELSE PROCdry
anlen% += INT(D+0.5)
nlen% += INT(L+0.5)
JCreord_from_to( Afrom%/size%, Ato%/size% )
JCreord_vector( X%, Y%, Z% )
JCreord_error( dX%, dY%, dZ% )
JComment(131)
JPROC

```

✓ this looks way, haven't we got Nfrom%, Nto%
we need to subtract 5490b if we want ~

```

FPROCdry
*****
dry : handle data for above water surveying style
Globals : PHI, PLUMB values of corrected clino reading and vertical plumb
          L, D, Z% true length, plan length and vertical component of leg
*****
I = FNreadno(nwidth%,2) - DPHI
JMB = FNreadno(nwidth%,7)
J  = SINRAD(PHI)
J  = COSRAD(PHI)
  = L * crp
  = L * srp
  = INT( D*srt +0.5)
  = INT( D*crt +0.5)
  = INT(Z+PLUMB+0.5)
L+PLUMB = 0 dX%=0:dY%=0:dZ%=0:ENDPROC
  = FNsquare( grade_spec( default_N_grade%, dP% ))/3
L<>0 dX += FNsquare( (X%/L) * grade_spec( default_N_grade%, dL% ))
+= FNsquare( Y% * grade_spec( default_N_grade%, dB% ))
+= FNsquare( PLUMB * grade_spec( default_N_grade%, dC% ))/2
+= FNsquare( Z * grade_spec( default_N_grade%, dC% )) * ( 0.5 + FNsquare(srt*crp
% = SQR( dX ) + 0.5)
  = FNsquare( grade_spec( default_N_grade%, dP% ))/3
L<>0 dY += FNsquare( (Y%/L) * grade_spec( default_N_grade%, dL% ))
+= FNsquare( X% * grade_spec( default_N_grade%, dB% ))
+= FNsquare( PLUMB * grade_spec( default_N_grade%, dC% ))/2
+= FNsquare( Z * grade_spec( default_N_grade%, dC% )) * ( 0.5 + FNsquare(crt*crp
% = SQR( dY ) + 0.5
  = FNsquare( grade_spec( default_N_grade%, dP% ))/3
+= FNsquare( (Z%/(L+PLUMB)) * grade_spec( default_N_grade%, dL% ))
+= FNsquare( D * grade_spec( default_N_grade%, dC% ))
% = SQR( dZ ) + 0.5
+= PLUMB
DPROC

```

```
FFNsquare( x ) = x * x
```

```

FPROCdiving
*****

```

```

diving : handle data for underwater surveying style
Globals : depthF, depthT depth of from and to stations
          Anew%, Afrom%, Ato% pointers to station names for 'last new', 'from' &
          Z, L depth change and true length of survey leg

```

```

*****
pthF = (FNreadno(nwidth%,7)-surface_depth%) * gauge_factor
pthT = (FNreadno(nwidth%,7)-surface_depth%) * gauge_factor

```

```

: Anew%=Afrom% depth=depthT:Anew%=Ato% ELSE IF Anew%=Ato% depth=depthF:Anew%=Afrom%
depthF:Anew%=Afrom% ELSE depth=depthT:Anew%=Ato%
= depthF-depthT
: ABSZ > L PROCthrowwobbler(3)
| actually, the above is possible, given the error margins on L and depth(s)
= SQR( L*L - Z*Z )
/ = INT( D * srt + 0.5 )
/ = INT( D * crt + 0.5 )
/ = INT( Z + 0.5 )
: L = 0 dX%=0:dY%=0:dZ%=0:ENDPROC
)diving% = grade_spec( default_D_grade%, dD% )
.diving = L - SQR( FNsquare(L) - FNsquare(dDdiving%) )
/ = FNsquare( grade_spec( default_D_grade%, dP% ))/3
: L<>0 dX += FNsquare( (X%/L) * grade_spec( default_D_grade%, dL% ))
/ += FNsquare( dLdiving * srt )
/ += FNsquare( Y% * grade_spec( default_N_grade%, dB% ))
dX% = SQR( dX ) + 0.5
/ = FNsquare( grade_spec( default_D_grade%, dP% ))/3
: L<>0 dY += FNsquare( (Y%/L) * grade_spec( default_D_grade%, dL% ))
/ += FNsquare( dLdiving * crt )
/ += FNsquare( X% * grade_spec( default_D_grade%, dB% ))
dY% = SQR( dY ) + 0.5
Z = FNsquare( grade_spec( default_D_grade%, dP% ))/3
Z += FNsquare( grade_spec( default_D_grade%, dD% ))
dZ% = SQR( dZ ) + 0.5
)DPROC

```

IFPROCradiolocation

```

*****

```

```

| radiolocation : process a radiolocation fix
|

```

```

*****

```

```

)OCskip( awidth%+1 )
.ror_h% = (FNreadno( nwidth%, 6 ) * 70711 + 0.5)
)OCskip( nwidth% )
.ror_v% = FNreadno( nwidth%, 5 )
)OCcomment(128)
)OCread_name( awidth%, text1% )
)OCread_name( awidth%, text2% )
: Origins% = 0 PROCinitial_origin(text1%,0,0,0)
)OClist(text1%,text2%)
)OCindicate_station
)OCskip( 3*nwidth% )
/ = FNreadno( nwidth%, 5 )
)OCrecord_from_to( Afrom%/size%, Ato%/size% )
)OCrecord_vector( 0; 0, Z% )
)OCrecord_error( error_h%, error_h%, error_v% )
)OCcomment(131)

```

oops! this depends on DPfix% i.e. it is $\Delta h \times 0.6 \times \sqrt{2}$

```

| Please note that radiolocations that do not specify a depth should NOT be
used until the Warren 88 treatment is implemented in full.
)DPROC

```

IFPROClist(from%,to%):*!LOCAL Qendsta%

```

*****

```

```

| list : check both survey stations for existence in our list. Add them if
they aren't already there, and return internal station numbers
Parms : from%, to% pointers to buffers holding 'from' and 'to' names
Globals : Afrom%, Ato% offsets to these names in the internal list
Nendsta% number of current highest station
Local : Qendsta% offset to next free space in internal station list

```

```

| BUG fix 1987.002 If both stations are undefined, they both got added to
the list in the same place - ie. 'from' station failed to be added.

```

```

*****
)endsta% = size% * Nendsta%
)from% = FNscan(from%)
)to% = FNscan(to%)
)Cscan( from%, Afrom%, Nfrom% )
)Cscan( to% , Ato% , Nto% )
)IF Afrom%<0 THEN
  Nfrom%<0 THEN
    Nendsta%+=1
    Qendsta%+=size%
    Afrom%=Qendsta%
    $(STA%Afrom%)=$from%
    STAX?(Afrom%+refs%)=1
  PROCadd_station( from%, Afrom%, Nfrom%, 1 )
  LSE
    STAX?(Afrom%+refs%)+=1
    Afrom%?refs%+=1
  NDIF
)IF Ato%<0 THEN
  Nto%<0 THEN
    Nendsta%+=1
    Ato%=Qendsta%+size%
    $(STA%Ato%)=$to%
    STAX?(Ato%+refs%)=1
  PROCadd_station( to%, Ato%, Nto%, 1 )
  LSE
    STAX?(Ato%+refs%)+=1
    Ato%?refs%+=1
  NDIF
DPROC

```

FFNscan(Aname%)

scan : search for a station name

1.40 : note that ARM version reverts to original BASIC string search. Since AB is very fast, this is still about twice as fast as machine code search in 6502 code ! Upgrade to ARM native code IS intended, however 1989.09.20 I apologise for the mucky way this gets out of the loop !

interface : in: Aname% points to string (8 chars) to search for
 Qendsta% is offset to last station added to STA% list
 out: return value is offset to found station, or
 return value -1 if not found.

CAL Asta%

R Asta% = STA%+Qendsta% TO STA% STEP -size%

F \$Asta%=\$Aname% Aname%=Asta%-STA%:Asta%=STA%-size%:NEXT:=Aname%

XT

1

FPROCscan(Aname%, RETURN Afound%, RETURN Nfound%)

scan : search for a station name

2.03 : returns address rather than pointer, also sets internal number
 uses extra slot at start of STA% area to speed search by cutting out
 bound checking - we will always find name, if its in the special
 slot, then it wasn't in the table.

Aendsta% is pointer to last station added to STAX list
out: Afound% return value is pointer to found station, or NULL
Nfound% is internal station number of found station or -1

```
CAL Asta%,Nsta%
ta% = Aendsta%
ta% = Nendsta%
ound% = NULL%
ound% = -1
find% = $Aname% ;!* guarantees we will always find it, saving range check
ILE $Aname%<>$Asta%
  Nsta%-=1
  Asta%-=size%
DWHILE
  Asta%=Afind% ENDPROC
ound% = Asta%
ound% = Nsta%
DPROC
```

```
FPROCadd_station( Aname%, RETURN Asta%, RETURN Nsta%, init_refs% )
ndsta% += 1
ndsta% += size%
endsta% = $Aname%
ta% = Aendsta%
ta% = Nendsta%
ta%?refs% = init_refs%
DPROC
```

Comments

FPROCunits

units : interpret FEET, METRIC or CONVERT keywords
Globals : convert current conversion factor if readno bit 1 is set

```
JCread_name(awidth%,text2%)
BE $text2% OF
```

```
4EN "FEET "
```

```
convert = 0.3048
conflag% = 4
PROCskip(nwidth%)
```

```
4EN "METRIC "
```

```
convert = 1
conflag% = 0
PROCskip(nwidth%)
```

```
4EN "CONVER ", "CONVERT "
```

```
convert = FNreadno(nwidth%,2)
conflag% = 4
```

```
7HERWISE
```

```
PROCthrowwobbler(101)
convert = 1
conflag% = 0
PROCskip(nwidth%)
```

} or should we leave unchanged?

```
7CASE
JCskip(3*nwidth%)
```

DPROC

FPROCtitle

```
title      : read title from input. Note that we could change title part way
Globals : title$
```

tle\$=FNrest

Popular FORTRAN SU 5 used to put the title at the top of each page, but we don't really know about pages and things on the output, so tough.

JCheading

DPROC

FPROCheading

```
heading : print a title. Note that we are in 40-column mode here, which is
          not really appropriate for the ARM
          eventually, we would put the title on the top of the window we
          are running in, and let RISCOS do the displaying
Globals : title$
```

INT"";LEFT\$(title\$,38)

(LENTtitle\$)>38 THEN

PRINT"";STRING\$(38,"=")'"";MID\$(title\$,39)'"";STRING\$((LENTtitle\$)-38,"=")

_SE

PRINT"";STRING\$(LENTtitle\$,"=")

VDIF

DPROC

FPROCnodes

nodes : find all the nodes in the survey

note that the initial origin is always treated as a node - this may be a prob

Globals : path\$, survey\$ pathname for all files, survey subdirectory.

Ohandle% handle for output to node file

J% count of stations

J%?refs% number of references to this station

Nendsta% station number of final station

on exit Nodecount% total number of nodes found

2.03 no longer assume origin is a node. This may be an error but easy to fix

CALL Asta%,Nsta%

filesize% = FTptr%

JCtime

MENT\$ = " "+STR\$~(!time%)+ " "+STR\$~(&FFF7FF00+time%?4)

LI("SAVE "+path\$+survey\$+".0 "+STR\$~(FTbuf%)+ " "+STR\$~(FTfilesize%)+COMMENT\$)

decoumt% = 0

R Asta% = STAX TO Aendsta% STEP size%

IF Asta%?refs%<>2 Nodecount%+=1

<T

INT"Finding ";Nodecount%; " nodes in survey :"

Nodes%(Nodecount%),Noderefs%(Nodecount%)

Nodes%(0) = 0:Noderefs%(0) = STAX?refs%:Node% = 1

de% = 0

R Nsta% = 0 TO Nendsta%

Asta% = STAX + Nsta% * size%

IF Asta%?refs%<>2 THEN

Nodes%(Node%) = Nsta%

```

Node%+=1
IF POS>32 PRINT
PRINT $Asta%
ENDIF
XT
Octime
MMENT$ = " "+STR$~(!time%)+ " "+STR$~(&FFF7FD00+time%?4)
CLI("SAVE "+path$+survey$+".N "+STR$~(STA%)+ " "+STR$~(size%*(Nendsta%+1))+COMME
DPROC

FPROCtidyup:LOCAL @%,J%
*****

tidyup : close files, print statistics and get user to input setup info for
        final stage of survey
Globals : Nendsta%, Nodecount%, planlen%, sumlen% - see caption printed with eac

*****
= &90A
OSE#0 :*! just to make sure (but doesn't do it on HostFS)
U12,26
RJX=0T01:PRINTCHR$141;CHR$134:NEXT
U28,0,23,39,2
ctor_time=(TIME-Start_Time%)/100
INT'"Processing to vector form in ";vector_time;" s"
INT'CHR$134;"STATISTICS : "
INT'"No. of survey stations ";Nendsta%+1
INT'"No. of nodes in survey ";Nodecount%
=&2020A
INT'"Total plan length "planlen%/DPfix%;" m"
INT'"Total survey length "sumlen%/DPfix%;" m"
UND1,-7,170,20
INT"DO YOU WANT OUTPUT PAGED ? ";;IF FNyn VDU14 ELSE VDU15
INT'"DO YOU WANT OUTPUT PRINTED ? ";;IF FNyn VDU2 ELSE VDU3
S
art_Time%=TIME
Ocheading
DPROC

FPROCcoords
*****

coords : calculate and display coordinates and closure errors
Globals : path$, survey$ path name for SU files, surey subdirectory
        Nendsta% number of stations to reserve space
        STA% base of internal station table
        initialX%... x,y,z coordinates of primary origin of survey
Locals : Arefs% pointer to reference count for each station

*****
CAL Arefs%
CAL@%:@%=&2020A
ile% = Csize%*(Nendsta%+1)
DIM Coords% Cfile%-1
tr%=0
A%?refs%=1
R Arefs%= STA%+size%+refs% TO STA%+area%+refs%-size% STEP size%
Arefs%=0
XT
CLI("SPOOL") :*! get rid of author's debugging file, if any.
CLI("SPOOL "+path$+survey$+".P")
Coords% = initialX%
ords%!4 = initialY%
ords%!8 = initialZ%

```

```

ptr% = 0
INT''Station      East/m      North/m      Height/m''-----
INT$STA%;TAB(cr%),initialX%/DPfix%,initialY%/DPfix%,initialZ%/DPfix%
ast% = -1
PEAT
ROCvector
TIL FTptr% = FTfilesize%
OCcotidy
CLI("SPDOL")
hell_String% = path$+survey$+".P"
Otime
ad% = &FFFFFFF00 + time%?4
ac% = !time%
S 8,1,Shell_String%,load%,exec%,,&33
MMENT$ = " "+STR$~(!time%)+ " "+STR$~(&FFF7FE00+time%?4)
CLI("SAVE "+path$+survey$+".C"+" "+STR$~(Coords%)+ " "+STR$~(Cfile%)+COMMENT$)
INT"Finished after ";(TIME-Start_Time%)/100;" seconds."
DPROC

FPROCvector
*****

vector : calculate a new coordinate or closure from vector in file, or warn
         user that his stations are undefined

Globals :
      Vft%          hi2bytes 'from', lo2bytes 'to' station no.
      xv%,yv%,zv% x,y,z vector for this survey leg
      Nfrom%, Nto% internal station numbers derived from Vft%
      Afrom%, Ato% pointers (NOT offsets) to station names in table
      Afrom%?refs%, Ato%?refs% specify whether each station is defined yet

*****

b% = FTbuf%!FTptr% : FTptr% += 4
% = Vbuf%!Vptr% : Vptr% += 4
% = Vbuf%!Vptr% : Vptr% += 4
% = Vbuf%!Vptr% : Vptr% += 4

rom% = Vft% >> 16
o% = Vft% AND 65535
rom% = STA% + size%*Nfrom%
o% = STA% + size%*Nto%

Nfrom% < 0 PROCneworg:ENDPROC

3E Afrom%?refs%+2*Ato%?refs% OF
WHEN 1
  PROCnewxyz(Nfrom%,Nto%, 1)
  PROCprint(Ato%)
WHEN 2
  PROCnewxyz(Nto%,Nfrom%,-1)
  PROCprint(Afrom%)
WHEN 3
  PROCclosed(Nfrom%,Nto%)
OTHERWISE
  PRINT$Afrom%;" TO ";$Ato%;" is undefined."
>CASE
>PROC

FPROCnewxyz(Nextant%,Ntofind%,LegSense%)
*****

newxyz : calculate new coordinate
      Nextant%      internal number of existing station
      Ntofind%      internal number of station being defined
      LegSense%     + or - unity. to allow for backsight

```


xa%, ya%, za% retained coordinate of last defined station

note that Nlast%,xa%,ya%,za% are a little obsolete now that everything is
in memory. This scheme was to reduce disc activity when the
coordinates were stored on disc.

```
CAL Alast%
  Nextant%(<>Nlast% THEN
OCAL Anew%
new% = Coords% + (Csize% * Nextant%)
a%   = !Anew%
a%   = Anew%!4
a%   = Anew%!8
DIF
ast% = Ntofind%
ast% = Coords% + (Csize% * Nlast%)
% += LegSense% * xv%
% += LegSense% * yv%
% += LegSense% * zv%
last% = xa%
ast%!4 = ya%
ast%!8 = za%
DPROC
```

FPROCneworg

neworg : define coordinates of alternate origin

```
CAL Alast%
ast% = Nto%
ast% = Coords% + (Csize% * Nlast%)
% = xv%
% = yv%
% = zv%
last% = xv%
ast%!4 = yv%
ast%!8 = zv%
INT"New origin : "
JCprint(Ato%)
DPROC
```

FPROCprint(Aname%)

print : print a set of coordinates
set 'defined' flag for this survey station
Alast% is address of station name/refcount, LOCAL to caller

```
ame%?refs% = 1
INT$Aname%;TAB(cr%),!Alast%/DPfix%,Alast%!4/DPfix%,Alast%!8/DPfix%
DPROC
```

FPROCclosed(Nextant%,Nclosing%)

closed : calculate and print a misclosure

```
CAL xnew%,ynew%,znew%
  Nextant% <> Nlast% THEN
JCAL Aextant%
```

```

a% = !Aextant%
a% = Aextant%!4
a% = Aextant%!8
DIF
ew% = xa% + xv%
ew% = ya% + yv%
ew% = za% + zv%
rev% = Coords% + (Csize% * Nclosing%)
% = !Aprev%
% = Aprev%!4
% = Aprev%!8
INT$Ato%:TAB(cr%),xnew%/DPfix%,ynew%/DPfix%,znew%/DPfix%
ew% -= xp%
ew% -= yp%
ew% -= zp%
INT"previous:"xp%/DPfix%,yp%/DPfix%,zp%/DPfix%
INT"MISCLOSES"xnew%/DPfix%,ynew%/DPfix%,znew%/DPfix%
ast%=-1 :*! actually this could be Nextant% if you want, xa%... are OK.
DPROC

FPROCcotidy:LOCAL @%
*****

cotidy : write out 'undefined' data for any undefined stations

Bug fix 1987.006 : Noted that this routine ENDPROCs rather early. Why ?
could be because the indirection in the loop fucks up because ? takes
precedence over * : now fixed.
Could be since (size%*Nsta%?refs%) should be (STA%?(size%*Nsta%+refs%)) !

*****
CAL Nsta%,Asta%
= &90A
R Nsta% = 0 TO Nendsta%
F (STA%?(size%*Nsta%+refs%))=0 THEN
Asta% = Coords% + (Csize% * .....
!Asta% = 0
!Asta%!4 = 0
!Asta%!8 = &80000000
NDIF
<T
>PROC

FPROCput_word(channel%,value%)
*****

*****
CALindex%
Rindex%=0 TO 3
PUT#channel%,value%AND&FF
blue% = value% >>> 8
<T
>PROC

)EFPROCbput(channel%,count%,!b%,b%!4,b%!8,b%!12)
*****

bput : invented March 1987, written August 1987 (!), abandoned at 1.61
procedure renders program independent of knowledge of exact location of the
resident integers which were previously used to make BPUT's simple.
This change is essential for portability to ARM or Communicator etc.
Now rendered irrelevant because we use memory, not a file !

```

```

LOCAL index%
count%=count%*4-1
FORindex%=0 TO count%
  BPUT#channel%,b%?index%
NEXT
ENDPROC

```

```

DEFPROCbget(channel%,count%,b%)

```

```

*****

```

```

bget : invented March 1987, written August 1987 (!), abandoned at 1.61
procedure renders program independent of knowledge of exact location of the
resident integers which were previously used to make BGET's simple.
Note: unlike PROCbput, PROCbget uses a pointer to the buffer
like bput, its now irrelevant

```

```

*****

```

```

LOCAL index%
count%=count%*4-1
FORindex%=0 TO count%
  b%?index%=BGET#channel%
NEXT
ENDPROC

```

```

FPROCunread

```

```

*****

```

```

There seems to be a bug in HostFS so that PTR# values occasionally have a
nybble corrupted to &F. This warns you its happening and gives a fairly
reliable fix, given the incidence of occurrence. Its actually overkill,
since it assumes that PTR#=. . . is a problem too, which it probably isn't.

```

```

*****

```

```

CAL A%,B%,C%,D%

```

```

= PTR#input%
= PTR#input%
R#input% = PTR#input% - 1      ;*! cannot use -=1 with pseudovariables
= PTR#input%
= PTR#input%

```

```

(A%<>B%) OR (C%<>D%) OR (C%<>A%-1) THEN
PRINT "Before ";~A%;" OR ";~B%
PRINT "After  ";~C%;" OR ";~D%
IF A%>B% A%=B%
REPEAT: PTR#input% = A%-1: UNTIL A%-1 = PTR#input%
>IF

```

```

>PROC

```

```

=FNensurePTR( handle% )

```

```

*****

```

```

This is to fix the same problem

```

```

*****

```

```

CAL A%,B%

```

```

= PTR#handle%
= PTR#handle%

```

```

A%>B% A%=B%

```

```

/

```

Shell code - part of this is to get environment string (which we want) and part is a general Shell to CLI routine (which we don't need). Trim out the redundant bits before releasing anything.

F PROCAssemble_Shell

This restricts you to Archimedes only - not even Brazil.
I think its OK on Arthur - try to avoid RISCOS specifics especially > 2.00

```
CAL A$,P%,O%,index%:DIM O% 1000,Shell_String% 256
S"OS_GetEnv" TO A$
ILELEFT$(A$,1)<>" "ANDLENA$<>0 A$=MID$(A$,2):ENDWHILE
ILELEFT$(A$,1)=" " A$=MID$(A$,2):ENDWHILE
ILERIGHT$(A$,1)=" " A$=LEFT$(A$):ENDWHILE
FNShell_String_UC(LEFT$(A$,5))="-QUIT"THEN
=MID$(A$,7):WHILELEFT$(A$,1)<>" "ANDLENA$<>0 A$=MID$(A$,2):ENDWHILE
ILELEFT$(A$,1)=" " A$=MID$(A$,2):ENDWHILE
DIF
  INSTR( A$, "@" ) A$=""
ell_Env$=A$
Rindex%=0TO2STEP2
=O%
OPT index%
hell_Call
OV R9,R0 ;save string address

OV R0,#0
OV R1,#0
OV R2,#0
OV R3,#2
OV R4,#0
OV R5,#0
OV R6,#0
OV R7,#0
WI "OS_SetEnv"
TMFD R13!,<R0-R7,R12> ; Save BASIC's environment
OV R0,#0
OV R1,#0
OV R2,#0
OV R3,#0
WI "OS_Control"
TMFD R13!,<R0-R3> ; Save BASIC's control
OV R0,#&10
OV R1,#0
OV R2,#0
WI "OS_ChangeEnvironment"
TMFD R13!,<R1,R2> ; Save BASIC's upcall handler

DR R3,[R14,#28] ; Magic useful value !
DR R3,[R8,R3] ; Address of current end of work space
TMFD R13!,<R3,R9,R14> ; Save parameters needed to restore world
JB R9,R13,#1024 ; Address of where to copy code to
OV R5,R9
DR R6,Shell_Job
DR R4,Shell_Job_End
hell_Copy
DR R2,[R6],#4 ; Copy the asm core program to the stack
TR R2,[R5],#4
MP R6,R4
JC Shell_Copy
```

```

Shell_Job
DR    R2,[R3,#-4]!      ; Now copy BASIC's workspace upstairs
TR    R2,[R9,#-4]!
MP    R3,#&8000
HI    Shell_Job
TMFD  R13!,<R9>

```

```

DR    R0,Shell_Exit
OV    R1,R9
IOV   R2,#0
IOV   R3,#2
IOV   R4,#0
OV    R5,#0
IOV   R6,#0
OV    R7,#0
WI    "OS_SetEnv"
DR    R0,Shell_Error
DR    R1,Shell_Buffer
OV    R2,#0
IOV   R3,#0
WI    "OS_Control"
OV    R0,#&10
DR    R1,Shell_UpCall
VN    R2,#0
WI    "OS_ChangeEnvironment"

TR    R13,Shell_Job_End
DR    R0,[R13,#4*2]
DD    R0,R0,R9
UB    R0,R0,#&8000
WI    "OS_CLI"

```

```

hell_Exit
OV    r0, #0

```

```

hell_Common
TR    r0, Shell_ErrorWord
DR    R13,Shell_Job_End
DMFD  R13!,<R8,R9,R10,R14>

```

```

DMFD  R13!,<R1-R2>
OV    R0,#&10
WI    "OS_ChangeEnvironment"
DMFD  R13!,<R0-R3>
WI    "OS_Control"
DMFD  R13!,<R0-R7, R12>
WI    "OS_SetEnv"

```

```

OV    R1,#&8000
hell_Restore
DR    R2,[R8],#4
TR    R2,[R11],#4
MP    R1,R9
CC    Shell_Restore
DR    r0, Shell_ErrorWord
EQ    r0,#0
ICEQS pc,r14,#&10000000
RRNES pc,r14,#&10000000

```

```

hell_UpCall ; Just stop them getting to BASIC's handler
JV    pc,r14

```

```

hell_Error
DR    r0, Shell_Buffer + 4

```

```

hell_Buffer ; Must be within copied area
QUD 0 ; You bastid !
QUD &12345678
QUS STRING$(128,CHR$13)
QUS STRING$(128,CHR$13)

hell_ErrorWord
QUD 0
hell_Job_End
QUD 0

XT
DPROC

F PROCShell(A$)
*****

PROCShell(program$) will execute the given program (anything!)
and then return to BASIC at the point after the function call.

*****
CALA%:A%=Shell_String%:$A%=A$:CALLShell_Call
DPROC

F FNShell_String_UC(A$)
*****

Function to convert string to upper case

*****
CALA%,Z%:A%=Shell_String%:$A%=A$
RZ%=0:LENA$-1:IF A%?Z%>=ASC"a" IF A%?Z%<=ASC"z" A%?Z%-=32
XT:=$A%

F FNShell_Array(A$(),A$)
*****

Function to split string to array of tokens and give number
NB original version did not use A$(0), but this one does.

*****
DIM A$(<>)<>1 ERROR 99,"Wrong size array for Shell_Array call"
CALA%,B%
=0:B%=INSTR(A$," ")
ILE B%<>0
A$(A%)=LEFT$(A$,B%-1):A%=MID$(A$,B%+1):A%+=1
WHILE LEFT$(A$,1)="" A%=MID$(A$,2):ENDWHILE
B%=INSTR(A$," ")
DWHILE
A$<>"" A$(A%)=A$:A%+=1
%

FFNSplit_path( RETURN path$ )
*****

take a file pathname like path$ = "fs::disc.$.dir.leaf" and return the leaf
while shortening path$ to "fs::disc.$.dir". if the path contains no dot
delimiter, then return the whole path as the function result, with path$=""

*****
CAL A%,B%

= INSTR( path$, "." )
A%=0 $Shell_String%=path$:path$="" $Shell_String%

```

```

FILE A%<>0
B% = A%
A% = INSTR( path$, ".", B%+1 )
IDWHILE
  Now B% points at last "." in path$
Shell_String% = MID$( path$, B%+1 )
lth$ = LEFT$( path$, B%-1 )
$Shell_String%

.FFNcheck_openinput
*****

take a directory pathname as path$.survey$. We expect to find our primary
input as path$.survey$.D, and any nested files in path$.Data. We check
the first requirement, but leave checking path$.Data until we actually
need a nested file (we normally expect one/some, but it is not required)

note that I am not happy with the new hierarchical structure for the data
files and something is going to get done to make it make more sense. This
may take the form of creating pseudo-applications so each survey can have
an icon, but still have a rational internal structure.

*****
CAL type%

IF path$<>" THEN
  $Shell_String% = path$
  SYS 8,5,Shell_String% TO type%
  CASE type% OF
    WHEN 0 : PRINT'"Cannot find directory ";path$:=0
    WHEN 1 : PRINT'"path$;" is not a directory":=0
    WHEN 2 : path$+=". "
    OTHERWISE : PRINT'"Object ";path$;" is neither file nor directory.":=0
  ENDCASE
ENDIF

$Shell_String% = path$+survey$
we already established that path$ is valid or null.

SYS 8,5,Shell_String% TO type%
CASE type% OF

  WHEN 0 : *! doesn't exist at all

    PRINT'"Cannot find directory ";$Shell_String%:=0

  WHEN 1 : *! its a file - but we wanted a directory

    PRINT'"$Shell_String%;" is a file, not a directory.":=0

  WHEN 2 : *! OK - all we need now is the primary data file :

    input% = OPENIN(path$+survey$+".D")
    IF input%=0 PRINT'"Unable to open ";path$+survey$+".D":=0

  OTHERWISE : *! should never happen

    PRINT'"Object ";$Shell_String%;" is neither file nor directory.":=0
ENDCASE

input%

PROCTime

```

This was SYS 7,1,time% under Brazil, but now we use extended RTC OSWORD :

```
time% = 3
/S 7,14,time%
IDPROC
```

IFPROCsave

```
| save : Overcome incompatibility between Archimedes BASIC and Brazil over
|       the SAVE keyword. This is ONLY relevant for Brazil !
|       actually, its quite handy under RISCOS too.
```

```
RELEASE=FNrelease
3CLI("save <CaveSurvey$Dir>.SU_"+STR$(INT(RELEASE))+"/"+RIGHT$("00"+STR$((100*REL
AGE)+" "+STR$~(TOP)+" 0 FFFFB00")
IDPROC
```

EM > BBC:\$.BASIC_lib.TubeTalk

IFPROCinitTUBEcommentary

```
| If you have no Host Podule, change Hostvdu: into whatever device you want
| to send debugging info to. ARM vdu is NOT recommended !
```

```
4 ERROR LOCAL: ON ERROR tube%=OPENOUT("null:"):ENDPROC
tube% = OPENOUT(debug_device$)
: tube% = 0 tube% = OPENOUT("null:")
IDPROC
```

IFPROCTUBEcomment(A\$, newline%)

```
PUT#tube%,A$;
: newline% BPUT#tube%,13:BPUT#tube%,10
IDPROC
```

IFPROCTUBEint(value%, hex%, newline%)

```
LOCAL val$
: hex% val$=STR$~(value%) ELSE val$=STR$(value%)
DOCTUBEcomment( val$, newline% )
IDPROC
```

IFPROCcloseTUBE

```
CLOSE#tube%
IDPROC
```

IFPROClie_in_state

```
DOCTUBEcomment(REPORT$+" at line "+STR$(ERL), TRUE )
DOCCloseTUBE
```

```
EM CLOSE#0 not guaranteed to close HostFS files owing to ADFS being current
EM we really need to handle handles cleanly, since we don't want, to have to
EM know what FS we have files open on etc.
```

```
DOCCease:REM this should do ? or is lie in state MEANT to leave file open ?
IDPROC
```